LAPORAN

LAYANAN IZIN DAN SAKIT: PENGAJUAN, BUKTI, PERSETUJUAN, DAN PENCATATAN KE PRESENSI

Anggota Kelompok :

Airil Nasbi (13020240075)

Zulfajrin (13020240084)

Muh Aslam (13020240088)

Ariel Satyawan Syarah (13020240017)

Andi Muhammad Dzaki Idrus (13020240306)

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS ILMU KOMPUTER

UNIVERSITAS MUSLIM INDONESIA

MAKASSAR

2026

DAFTAR ISI

DAFTAR ISI i
BAB I PENDAHULUAN 1
1.1 Ringkas Layanan 1
1.2 Identifikasi Masalah 1
1.3 Identifikasi Stakeholder 2
1.4 Architecturally Significant Requirements (ASR) 3
1.5 Karakteristik Pendorong (Architectural Drivers) 3
BAB II SKENARIO 5
2.1 Skenario 1 (Performance) 5
2.2 Skenario 2 (Security) 5
2.3 Skenario 3 (Reliability) 6
2.4 Ringkasan Skenario 6
BAB III TAKTIK DAN POLA 8
3.1 Taktik Arsitektur (U3) 8
3.2 Pola Arsitektur (U4) 9
3.4 Perbedaan Gaya Arsitektur dan Pola Arsitektur 10
BAB IV DESAIN 11
4.1 Diagram Konteks 11
4.2 Diagram Arsitektur 12
4.3 UML Class Diagram 13
4.4 Prinsip Desain yang Diterapkan 14
BAB V EVALUASI DAN ADR 15
5.1 Pohon Utilitas (Utility Tree) 15
5.2 Analisis Trade-off Berbobot 16

5.3 Analisis ala ATAM 17

5.4 Apakah Masalah Utama Teratasi? 18

5.5 Architecture Decision Record (ADR)..... 19

BAB VI PENUTUP 22

6.1 Kesimpulan 22

6.2 Saran.....

DAFTAR PUSTAKA.....23

BAB I
PENDAHULUAN

1.1 Ringkas Layanan

Layanan izin dan sakit mahasiswa merupakan layanan akademik yang digunakan untuk mengelola ketidakhadiran mahasiswa dalam kegiatan perkuliahan karena alasan tertentu, seperti sakit, keperluan keluarga, atau kepentingan lainnya.

Pada proses yang berjalan saat ini, mahasiswa umumnya menghubungi dosen melalui WhatsApp atau menyerahkan surat izin secara langsung. Dosen kemudian memeriksa alasan dan bukti yang diberikan sebelum menentukan apakah pengajuan diterima atau ditolak.

Proses tersebut masih memiliki beberapa kendala, seperti keterlambatan respons dosen, bukti pengajuan yang berisiko hilang atau tertumpuk, ketidakjelasan status pengajuan, serta kemungkinan ketidaksesuaian pencatatan izin dengan data presensi.

Oleh karena itu, dirancang sebuah layanan digital yang dapat mengintegrasikan proses pengajuan izin dan sakit, pengunggahan bukti pendukung, pemeriksaan oleh dosen, pemberian keputusan, notifikasi status, serta pencatatan pengajuan yang disetujui ke sistem presensi.

Tujuan layanan

1. Mempermudah mahasiswa mengajukan izin dan sakit.
2. Mempermudah dosen memeriksa dan memberikan keputusan.
3. Mengurangi risiko kehilangan bukti pengajuan.
4. Memberikan informasi status pengajuan kepada mahasiswa.
5. Mempermudah pencatatan izin dan sakit ke sistem presensi.
6. Mengurangi kesalahan pencatatan dan duplikasi data.

1.2 Identifikasi Masalah

Berdasarkan pengalaman pengguna dan permasalahan pada proses pengajuan izin dan sakit, masalah utama yang diangkat adalah:

Belum terintegrasinya proses pengajuan izin dan sakit mahasiswa dengan pencatatan presensi sehingga proses pengajuan, pemeriksaan, pemantauan status, dan pencatatan masih berpotensi dilakukan secara manual.

Masalah tersebut dapat menimbulkan beberapa dampak:

1. Mahasiswa harus menghubungi dosen secara langsung untuk mengajukan izin.
2. Pengajuan dapat tertumpuk di antara pesan WhatsApp lainnya.
3. Mahasiswa kesulitan mengetahui apakah pengajuan telah disetujui atau ditolak.
4. Bukti pengajuan berisiko hilang atau sulit ditemukan kembali.
5. Pencatatan izin ke presensi berpotensi terlambat atau tidak sesuai.

1.3 Identifikasi Stakeholder

Stakeholder merupakan pihak yang memiliki kepentingan atau terlibat dalam layanan izin dan sakit mahasiswa.

Untuk rancangan ini digunakan empat stakeholder utama sesuai batas minimal tugas.

Kode	Stakeholder	Kelompok	Peran dalam layanan	Kebutuhan utama
ST01	Mahasiswa	Pengguna utama	Mengajukan izin dan sakit serta memantau status.	Formulir mudah digunakan, unggah bukti, informasi status.
ST02	Dosen	Pemberi persetujuan	Memeriksa, menyetujui, atau menolak pengajuan.	Daftar pengajuan, bukti pendukung, fitur keputusan.
ST03	Admin Akademik	Pengelola administrasi	Membantu pengelolaan dan pemantauan data layanan.	Data tersimpan rapi dan laporan pengajuan.

ST04	Operator/Pengelola Presensi	Pengelola presensi	Memastikan data izin yang disetujui tercatat pada presensi.	Data pengajuan yang akurat dan integrasi presensi.

1.4 Architecturally Significant Requirements (ASR)

ASR merupakan kebutuhan sistem yang memiliki pengaruh besar terhadap keputusan arsitektur.

Berikut tiga ASR utama yang digunakan:

Kode	Kebutuhan arsitektur	Atribut kualitas	Alasan
ASR01	Sistem harus mampu memproses pengajuan dan menampilkan status secara cepat.	Performance	Mengurangi waktu tunggu mahasiswa.
ASR02	Sistem harus melindungi data mahasiswa dan dokumen bukti pengajuan dari akses tanpa izin.	Security	Menjaga kerahasiaan data.
ASR03	Sistem harus memastikan pengajuan yang disetujui tetap tersimpan dan tercatat pada presensi meskipun terjadi gangguan.	Reliability	Menghindari kehilangan dan duplikasi data.

1.5 Karakteristik Pendorong (Architectural Drivers)

Karakteristik pendorong adalah kebutuhan utama yang memengaruhi pemilihan arsitektur sistem.

Kode	Karakteristik pendorong	Penjelasan	Prioritas
AD01	Kecepatan respons	Sistem harus memproses pengajuan dan menampilkan status dengan cepat.	Tinggi
AD02	Keamanan data	Sistem harus menjaga informasi dan dokumen mahasiswa.	Tinggi
AD03	Keandalan integrasi presensi	Data persetujuan harus tetap tersimpan dan dapat dikirim ke sistem presensi.	Tinggi

BAB II
SKENARIO ATRIBUT KUALITAS

2.1 Skenario 1 – Performance (Kecepatan Respons)

Komponen	Penjelasan
Source	Mahasiswa
Stimulus	Mahasiswa mengirimkan pengajuan izin atau sakit.
Environment	Sistem beroperasi dalam kondisi normal dengan banyak pengguna aktif.
Artifact	Modul pengajuan izin dan sakit.
Response	Sistem memvalidasi data, menyimpan pengajuan, dan menampilkan nomor serta status pengajuan.
Response Measure	95% pengajuan mendapatkan respons maksimal 3 detik.

2.2 Skenario 2 – Security (Keamanan Data)

Komponen	Penjelasan
Source	Pengguna yang tidak memiliki hak akses.
Stimulus	Pengguna mencoba membuka dokumen bukti izin atau sakit milik mahasiswa lain.

Environment	Sistem sedang beroperasi dan pengguna sudah masuk ke aplikasi.
Artifact	Modul autentikasi, otorisasi, dan penyimpanan dokumen.
Response	Sistem memeriksa hak akses dan menolak permintaan yang tidak memiliki izin.
Response Measure	100% permintaan akses tanpa izin ditolak dalam pengujian keamanan yang telah disiapkan.

2.3 Skenario 3 – Reliability (Keandalan Integrasi Presensi)

Komponen	Penjelasan
Source	Dosen
Stimulus	Dosen menyetujui pengajuan izin atau sakit mahasiswa.
Environment	Sistem berjalan, tetapi sistem presensi mengalami gangguan sementara.
Artifact	Modul persetujuan dan integrasi presensi.
Response	Sistem menyimpan keputusan dosen dan memasukkan permintaan pencatatan ke antrean untuk dikirim kembali ketika sistem presensi tersedia.
Response Measure	100% persetujuan tersimpan dan masuk antrean. Sebanyak 95% pencatatan berhasil maksimal 5 menit setelah sistem presensi kembali tersedia.

2.4 Ringkasan Skenario

Kode	Atribut kualitas	Target keberhasilan
QS01	Performance	95% respons maksimal 3 detik.

QS02	Security	100% akses tanpa izin ditolak dalam pengujian.
QS03	Reliability	100% persetujuan tersimpan dan masuk antrean; 95% pencatatan maksimal 5 menit setelah presensi pulih.

BAB III
TAKTIK DAN POLA ARSITEKTUR

3.1 Taktik Arsitektur (U3)

A. Taktik untuk QS01 – Performance

Nama taktik: Caching dan Database Indexing

- Cara kerja: Sistem menggunakan caching untuk data yang sering diakses, seperti daftar mata kuliah dan informasi pengguna. Database menggunakan indeks pada kolom pencarian seperti NIM, tanggal pengajuan, dan status.
- Tujuan: Mempercepat proses pengambilan dan pengolahan data.
- Harga yang dibayar: Membutuhkan tambahan memori dan pengelolaan agar data cache tetap sesuai dengan data terbaru.

B. Taktik untuk QS02 – Security

Nama taktik: Authentication dan Role-Based Access Control (RBAC)

- Cara kerja: Pengguna harus login terlebih dahulu. Setelah berhasil login, sistem menentukan hak akses berdasarkan peran pengguna, yaitu mahasiswa, dosen, dan admin.
- Tujuan: Mencegah pengguna membuka data atau dokumen yang bukan menjadi haknya.
- Harga yang dibayar: Membutuhkan pengelolaan akun, peran, dan hak akses yang lebih terstruktur. Proses pemeriksaan hak akses juga menambah sedikit waktu pemrosesan.

C. Taktik untuk QS03 – Reliability

Nama taktik: Message Queue dan Retry Mechanism

- Cara kerja: Ketika dosen menyetujui pengajuan, sistem menyimpan keputusan ke database dan membuat pesan pencatatan untuk dikirim ke sistem presensi.

Jika sistem presensi mengalami gangguan, pesan tetap disimpan dalam antrean dan dikirim ulang secara otomatis ketika layanan kembali tersedia.

- Tujuan: Menjaga agar data persetujuan tidak hilang dan tetap dapat dicatat ke sistem presensi.
- Harga yang dibayar: Membutuhkan komponen antrean pesan, mekanisme percobaan ulang, serta pengelolaan status pesan untuk mencegah pencatatan ganda.

3.2 Pola Arsitektur (U4)

A. Layered Architecture (Pola Berlapis)

Pola struktural

Layered Architecture merupakan pola yang membagi sistem menjadi beberapa lapisan berdasarkan tanggung jawab masing-masing.

Alasan pemilihan:

Pola ini dipilih karena sistem memiliki beberapa proses berbeda yang membutuhkan pembagian tanggung jawab agar pengembangan lebih teratur.

Kelebihan:

- Struktur sistem lebih terorganisasi.
- Setiap lapisan memiliki tanggung jawab yang jelas.
- Mempermudah pemeliharaan sistem.
- Perubahan antarmuka tidak harus mengubah seluruh proses bisnis.

Kekurangan:

- Komunikasi melewati beberapa lapisan dapat menambah waktu pemrosesan.
- Perubahan pada lapisan tertentu dapat memengaruhi lapisan lain jika batas tanggung jawab tidak dijaga.

B. Event-Driven Architecture (Arsitektur Berbasis Peristiwa)

Pola komunikasi

Event-Driven Architecture merupakan pola arsitektur yang memungkinkan komponen sistem berkomunikasi berdasarkan peristiwa atau kejadian tertentu.

Alasan pemilihan:

Pola ini dipilih agar proses persetujuan, pengiriman notifikasi, dan pencatatan presensi tidak harus dilakukan secara bersamaan dalam satu proses.

Kelebihan:

- Komponen sistem tidak terlalu bergantung satu sama lain.
- Proses dapat berjalan secara asinkron.
- Gangguan pada satu layanan tidak harus menghentikan seluruh layanan.
- Mempermudah penambahan komponen baru.

Kekurangan:

- Membutuhkan pengelolaan antrean dan peristiwa.
- Proses pelacakan kesalahan lebih kompleks.
- Membutuhkan mekanisme untuk mencegah pesan diproses lebih dari sekali.

3.4 Perbedaan Gaya Arsitektur dan Pola Arsitektur

Gaya Arsitektur	**Pola Arsitektur**
Merupakan prinsip atau pendekatan umum dalam merancang keseluruhan struktur sistem.	Merupakan solusi yang lebih spesifik untuk menyelesaikan masalah desain tertentu dalam sistem.

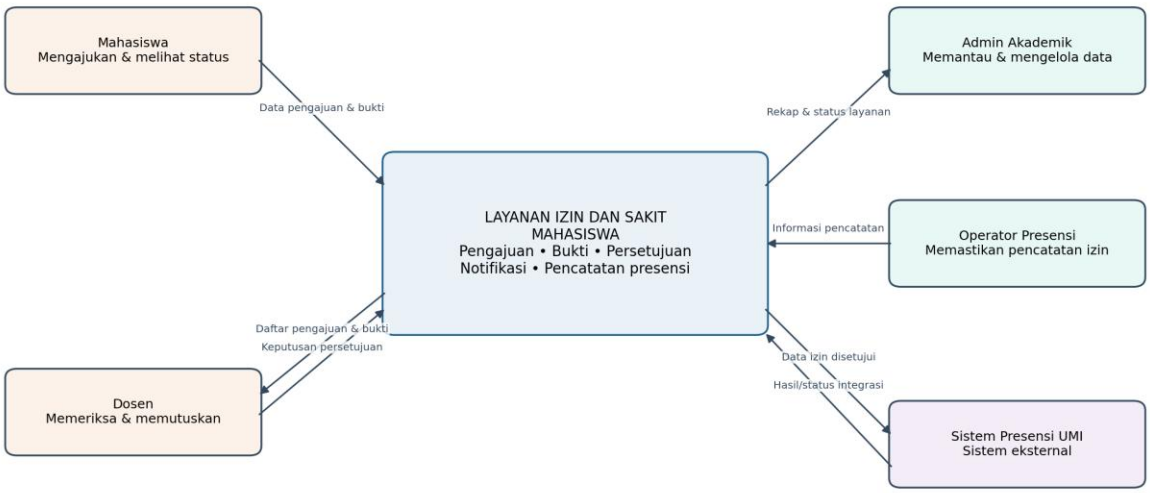
BAB IV

DESAIN

4.1 Diagram Konteks

Diagram konteks menunjukkan hubungan antara layanan izin dan sakit mahasiswa dengan pihak-pihak yang berinteraksi di luar sistem. Sistem menjadi pusat pengelolaan pengajuan, dokumen bukti, persetujuan, notifikasi, dan pencatatan izin ke presensi.

Gambar 4.1 Diagram Konteks Layanan Izin dan Sakit Mahasiswa



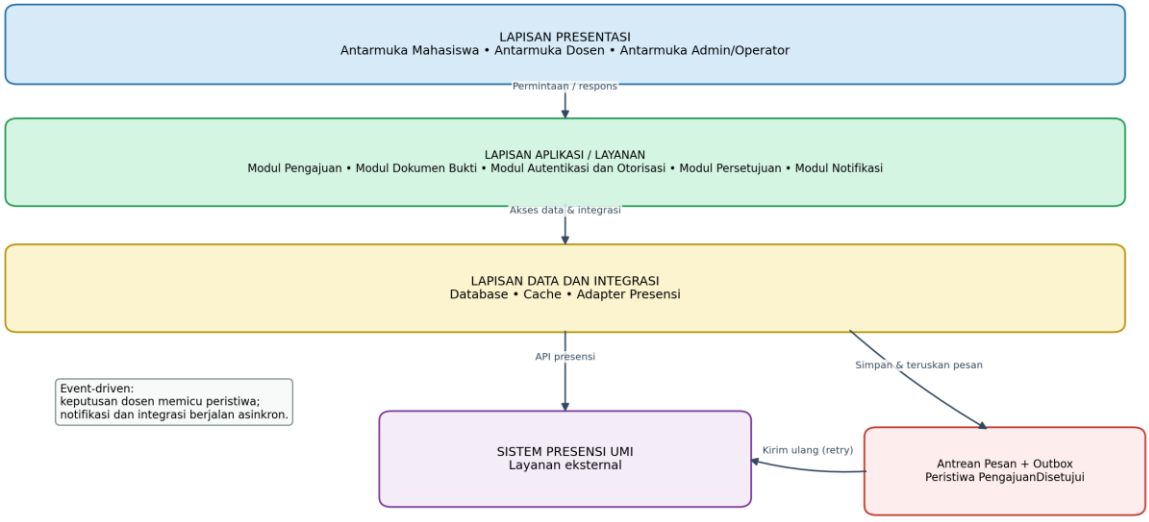
Gambar 4.1 Diagram Konteks Layanan Izin dan Sakit Mahasiswa

Berdasarkan Gambar 4.1, Mahasiswa mengirimkan data pengajuan dan dokumen bukti, kemudian menerima informasi nomor serta status pengajuan. Dosen menerima daftar pengajuan dan bukti untuk diperiksa, lalu mengirimkan keputusan persetujuan atau penolakan. Admin Akademik memantau dan mengelola data layanan, sedangkan Operator/Pengelola Presensi memastikan izin yang disetujui tercatat dengan benar. Layanan juga bertukar data dengan Sistem Presensi UMI sebagai sistem eksternal.

4.2 Diagram Arsitektur

Rancangan menggunakan gaya Modular Monolith, yaitu seluruh modul utama berada dalam satu unit penerapan tetapi dipisahkan berdasarkan tanggung jawab. Di dalamnya diterapkan pola Layered Architecture untuk membagi sistem menjadi lapisan presentasi, aplikasi/layanan, serta data dan integrasi. Pola Event-Driven Architecture digunakan agar proses lanjutan, seperti pengiriman notifikasi dan pencatatan presensi, dapat berjalan berdasarkan peristiwa tanpa harus menunggu seluruh proses selesai.

Gambar 4.2 Diagram Arsitektur Modular Monolith dengan Layered dan Event-Driven



Gambar 4.2 Diagram Arsitektur Modular Monolith dengan Layered dan Event-Driven

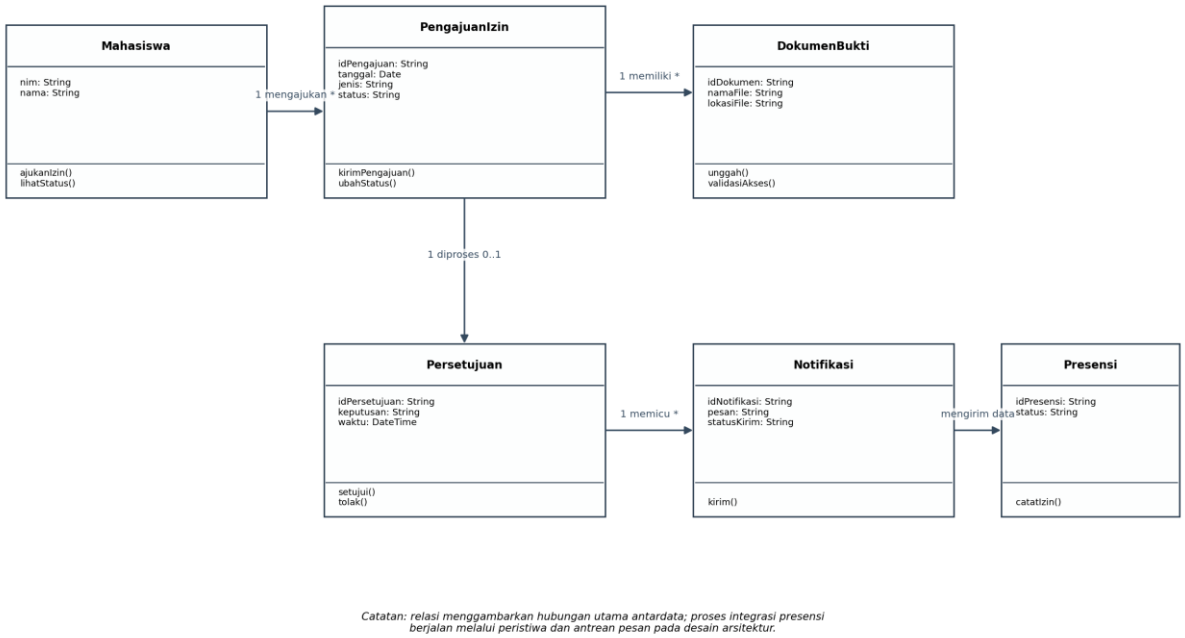
Lapisan presentasi menjadi tempat mahasiswa, dosen, admin akademik, dan operator presensi berinteraksi dengan sistem. Lapisan aplikasi/layanan memuat Modul Pengajuan, Modul Dokumen Bukti, Modul Autentikasi dan Otorisasi, Modul Persetujuan, serta Modul Notifikasi. Lapisan data dan integrasi mengelola Database, Cache, dan Adapter Presensi. Ketika dosen menyetujui pengajuan, sistem menyimpan keputusan beserta pesan pada mekanisme outbox. Antrean Pesan meneruskan peristiwa PengajuanDisetujui ke Adapter Presensi. Jika sistem presensi belum tersedia, pesan dapat dikirim ulang menggunakan mekanisme retry.

Pemisahan lapisan membuat tanggung jawab komponen lebih jelas, sedangkan komunikasi berbasis peristiwa mengurangi ketergantungan langsung antara proses persetujuan, notifikasi, dan pencatatan presensi.

4.3 UML Class Diagram

UML Class Diagram berikut menggambarkan struktur data utama yang mendukung proses layanan. Kelas Mahasiswa berhubungan dengan PengajuanIzin karena seorang mahasiswa dapat membuat beberapa pengajuan. Setiap pengajuan dapat memiliki dokumen bukti dan diproses melalui Persetujuan. Keputusan persetujuan dapat memicu Notifikasi serta pengiriman data izin ke bagian Presensi.

Gambar 4.3 UML Class Diagram Layanan Izin dan Sakit



Gambar 4.3 UML Class Diagram Layanan Izin dan Sakit

Keterangan kelas:

- **Mahasiswa:** menyimpan identitas mahasiswa yang mengajukan izin atau sakit.
- **PengajuanIzin:** menyimpan informasi pengajuan, tanggal, jenis izin, dan status.

- **DokumenBukti:** menyimpan informasi berkas pendukung yang diunggah mahasiswa.
- **Persetujuan:** menyimpan keputusan dosen, baik disetujui maupun ditolak.
- **Notifikasi:** menyimpan pesan dan status pengiriman informasi kepada pengguna.
- **Presensi:** mewakili pencatatan izin yang telah disetujui pada sistem presensi.

4.4 Prinsip Desain yang Diterapkan

Separation of Concerns (Pemisahan Tanggung Jawab). Setiap lapisan dan modul memiliki tugas yang jelas. Modul Pengajuan menangani pengajuan, Modul Dokumen Bukti menangani berkas, Modul Persetujuan menangani keputusan dosen, dan Modul Notifikasi menangani pemberitahuan. Pemisahan ini memudahkan pengembangan serta pemeliharaan.

Least Privilege (Hak Akses Minimum). Setiap pengguna hanya memperoleh hak akses sesuai perannya. Mahasiswa hanya mengelola pengajuannya sendiri, dosen memeriksa pengajuan yang menjadi kewenangannya, sedangkan admin dan operator presensi mengakses fungsi sesuai tugas masing-masing. Pemeriksaan dilakukan melalui Modul Autentikasi dan Otorisasi dengan RBAC.

Loose Coupling (Ketergantungan Rendah). Proses persetujuan tidak bergantung langsung pada ketersediaan sistem presensi. Peristiwa PengajuanDisetujui dikirim melalui Antrean Pesan dan Adapter Presensi. Dengan cara ini, gangguan sementara pada sistem presensi tidak menghilangkan keputusan dosen dan pengiriman dapat dicoba kembali.

Dengan menerapkan ketiga prinsip tersebut, rancangan diharapkan memiliki pembagian tanggung jawab yang jelas, perlindungan data berdasarkan peran, serta integrasi presensi yang lebih andal. Nama komponen dan pola pada diagram ini disesuaikan dengan rancangan pada Bab II, Bab III, dan evaluasi pada Bab V.

BAB V
EVALUASI DAN ADR

5.1 Pohon Utilitas (Utility Tree)

Pohon utilitas menjabarkan tujuan kualitas sistem menjadi skenario konkret. Setiap skenario diberi peringkat (Importance, Difficulty) dengan skala H (High), M (Medium), dan L (Low). Importance menyatakan seberapa penting skenario bagi pemangku kepentingan, sedangkan Difficulty menyatakan seberapa sulit skenario dicapai secara arsitektural.

Tabel 5.1 Pohon utilitas layanan izin dan sakit mahasiswa

Atribut kualitas	Penyempurnaan	Skenario dan ukuran respons	(I, D)	Kaitan masalah utama
Utility › Performance	Kecepatan pengajuan dan tampilan status	QS01: 95% pengajuan mendapat respons maksimal 3 detik, lengkap dengan nomor dan status pengajuan.	(H, M)	Menjawab status yang tidak jelas dan pengajuan yang tertumpuk
Utility › Security	Perlindungan dokumen bukti	QS02: 100% permintaan akses dokumen tanpa izin ditolak dalam pengujian keamanan.	(H, L)	Menjaga bukti pengajuan tetap aman dan tidak tercecer
Utility › Reliability	Integrasi persetujuan ke presensi	QS03: 100% persetujuan tersimpan dan masuk antrean; 95% pencatatan berhasil maksimal 5 menit setelah presensi pulih.	(H, H)	Menjawab langsung masalah utama: izin belum terintegrasi dengan presensi

QS03 mendapat peringkat tertinggi (H, H) karena skenario ini menjawab langsung masalah utama dan paling sulit dicapai, sebab bergantung pada sistem presensi yang

berada di luar kendali layanan. Karena itu, QS03 menjadi fokus utama pada analisis ATAM di subbab 5.3.

5.2 Analisis Trade-off Berbobot

Dua alternatif arsitektur dibandingkan. Alternatif A adalah Modular Monolith dengan pola Layered Architecture dan Event-Driven Architecture (rancangan yang dipilih). Alternatif B adalah Microservices, dengan layanan terpisah untuk pengajuan, persetujuan, notifikasi, dan integrasi presensi. Penilaian memakai skala 1 sampai 5 (5 = paling baik). Bobot kriteria disusun dari prioritas pada pohon utilitas; Reliability integrasi presensi diberi bobot terbesar karena terkait langsung dengan masalah utama.

Tabel 5.2 Perbandingan berbobot Modular Monolith dan Microservices

Kriteria	Bobot	Skor A	Skor B	A (bobot × skor)	B (bobot × skor)
Performance (QS01)	15%	4	3	0,60	0,45
Security (QS02)	20%	4	4	0,80	0,80
Reliability integrasi presensi (QS03)	30%	4	5	1,20	1,50
Kemudahan pengembangan dan pemeliharaan	15%	5	2	0,75	0,30
Biaya dan kompleksitas operasional	10%	5	3	0,50	0,30
Skalabilitas	10%	3	5	0,30	0,50
Total	100%			4,15	3,85

Pembahasan. Modular Monolith unggul (4,15 berbanding 3,85) terutama pada kemudahan pengembangan dan biaya operasional. Hal ini sesuai dengan kondisi layanan, yaitu tim pengembang kecil dan skala pengguna sebatas satu kampus. Pada Microservices, setiap pemanggilan antarlayanan melewati jaringan sehingga latensi bertambah (Performance 3), tetapi isolasi layanan membuat Reliability dan Skalabilitas

lebih baik. Selisih nilai sebesar 0,30 menunjukkan bahwa pilihan ini cukup sensitif terhadap bobot Reliability; hal tersebut dibahas sebagai sensitivity point pada subbab 5.3. Karena Event-Driven Architecture dan antrean pesan sudah dipakai di dalam Modular Monolith, sebagian besar manfaat Reliability dari Microservices tetap diperoleh tanpa biaya operasionalnya.

5.3 Analisis ala ATAM

Analisis dilakukan terhadap skenario prioritas tinggi pada pohon utilitas. Nama komponen mengikuti diagram pada Bab IV: Modul Pengajuan, Modul Dokumen Bukti, Modul Autentikasi dan Otorisasi, Modul Persetujuan, Modul Notifikasi, Adapter Presensi, Antrean Pesan, Cache, dan Database.

Tabel 5.3 Hasil analisis ala ATAM

Jenis temuan	Skenario	Uraian	Dampak dan tindak lanjut
Risiko 1	QS03	Antrean Pesan yang berjalan di dalam proses aplikasi yang sama dapat kehilangan pesan jika server mati sebelum pesan terkirim ke Adapter Presensi.	Persetujuan tersimpan, tetapi pencatatan presensi tidak pernah terkirim. Mitigasi: simpan pesan pada tabel outbox di Database dalam satu transaksi dengan keputusan dosen (lihat ADR-002).
Risiko 2	QS03	Kontrak antarmuka (API) sistem presensi UMI belum terverifikasi, termasuk apakah permintaan yang sama dua kali diterima dengan aman.	Pengiriman ulang dapat menyebabkan pencatatan ganda. Mitigasi: kunci idempotensi per pengajuan dan pengujian integrasi dengan pengelola presensi sebelum diterapkan.
Non-risiko	QS02	Seluruh akses dokumen melewati Modul Autentikasi dan Otorisasi pada lapisan layanan (Layered Architecture), sehingga pemeriksaan peran dilakukan di satu tempat.	Tidak ada jalur yang melewati pemeriksaan hak akses; QS02 dinilai dapat dipenuhi.
Sensitivity point	QS03	Interval dan jumlah percobaan ulang (retry) pada Adapter Presensi	Interval terlalu panjang melampaui 5 menit; terlalu pendek membebani sistem

Jenis temuan	Skenario	Uraian	Dampak dan tindak lanjut
		menentukan tercapainya target 95% pencatatan maksimal 5 menit.	presensi yang baru pulih. Nilai awal diusulkan 30 detik dengan jeda bertahap, lalu disetel melalui pengujian.
Trade-off point	QS01 dan QS02	Pemeriksaan hak akses (RBAC) dan lapisan berjenjang menambah waktu pemrosesan setiap permintaan, sedangkan Cache mempercepat respons tetapi dapat menyimpan data peran atau status yang sudah usang.	Keamanan dan kecepatan saling menarik. Mitigasi: Cache hanya untuk data referensi, sedangkan keputusan hak akses dan status pengajuan selalu dibaca dari Database.

5.4 Apakah Masalah Utama Teratasi?

Masalah utama: belum terintegrasinya proses pengajuan izin dan sakit mahasiswa dengan pencatatan presensi, sehingga pengajuan, pemeriksaan, pemantauan status, dan pencatatan masih berpotensi dilakukan secara manual.

Dampak yang dikemukakan pada Bab I dipetakan terhadap rancangan sebagai berikut.

Tabel 5.4 Pemetaan dampak masalah terhadap rancangan

Dampak pada kondisi saat ini	Penanganan pada rancangan	Skenario/pola
Pengajuan tertumpuk di WhatsApp dan harus menghubungi dosen langsung	Pengajuan dikirim melalui Modul Pengajuan dan muncul pada daftar dosen	QS01, Layered Architecture
Mahasiswa sulit mengetahui status	Nomor dan status ditampilkan segera; Modul Notifikasi mengirim pemberitahuan setelah keputusan	QS01, Event-Driven Architecture
Bukti berisiko hilang atau sulit ditemukan	Bukti disimpan terpusat pada Modul Dokumen Bukti dengan pembatasan akses berbasis peran	QS02, RBAC

Dampak pada kondisi saat ini	Penanganan pada rancangan	Skenario/pola
Pencatatan ke presensi terlambat atau tidak sesuai	Persetujuan memicu peristiwa PengajuanDisetujui, dikirim lewat Antrean Pesan dan Adapter Presensi dengan retry dan kunci idempotensi	QS03, Event-Driven Architecture, Message Queue

Kesimpulan. Masalah utama dinilai teratasi pada tingkat rancangan, karena setiap dampak memiliki mekanisme arsitektural yang menjawabnya dan masalah inti (pencatatan ke presensi) ditangani oleh skenario QS03 yang berperingkat tertinggi. Pernyataan ini belum didukung pengukuran pada sistem berjalan. Ketercapaian penuh bergantung pada dua syarat dari analisis ATAM, yaitu pesan antrean disimpan secara tahan lama (Risiko 1) dan API presensi terverifikasi (Risiko 2). Pembuktian akhir memerlukan pengujian beban untuk QS01, pengujian keamanan untuk QS02, dan simulasi gangguan presensi untuk QS03.

5.5 Architecture Decision Record (ADR)

Tiga keputusan arsitektur penting dicatat dalam format ADR (Title, Status, Context, Decision, Consequences). Ringkasannya pada Tabel 5.5, sedangkan catatan lengkapnya ada pada Lampiran A.

Tabel 5.5 Ringkasan ADR

Kode	Keputusan	Status	Dasar
ADR-001	Memilih Modular Monolith dibanding Microservices	Diterima	Tabel 5.2; Layered Architecture
ADR-002	Integrasi presensi melalui Antrean Pesan, outbox, retry, dan kunci idempotensi	Diterima	QS03; Event-Driven Architecture;

Kode	Keputusan	Status	Dasar
			Message Queue dan Retry
ADR-003	RBAC di lapisan layanan dan Cache terbatas pada data referensi	Diterima	QS01, QS02; RBAC; Caching dan Indexing

ADR-001: Memilih Modular Monolith dibanding Microservices

Title	Memilih Modular Monolith dibanding Microservices
Status	Diterima (November 2026)
Context	Layanan dikembangkan oleh tim kecil untuk lingkup satu kampus. Alternatif Microservices dan Modular Monolith dibandingkan pada Tabel 5.2 dengan hasil 3,85 berbanding 4,15. Layanan tetap harus mendukung integrasi presensi yang andal (QS03).
Decision	Sistem dibangun sebagai satu unit penerapan (deployment) yang dibagi menjadi modul: Modul Pengajuan, Modul Dokumen Bukti, Modul Autentikasi dan Otorisasi, Modul Persetujuan, dan Modul Notifikasi. Modul berkomunikasi lewat antarmuka yang jelas dan peristiwa, sehingga dapat dipisah menjadi layanan sendiri jika diperlukan.
Consequences	Positif: pengembangan, pengujian, dan operasi lebih sederhana; tidak ada latensi jaringan antarmodul. Negatif: skalabilitas per modul terbatas, dan kegagalan satu modul dapat memengaruhi seluruh proses. Batas antarmodul harus dijaga agar tidak saling bergantung langsung.

ADR-002: Integrasi presensi melalui Antrean Pesan, outbox, retry, dan kunci idempotensi

Title	Integrasi presensi melalui Antrean Pesan, outbox, retry, dan kunci idempotensi

Status	Diterima (November 2026)
Context	Masalah utama layanan adalah pencatatan izin ke presensi yang belum terintegrasi. Skenario QS03 mengharuskan 100% persetujuan tersimpan dan 95% pencatatan berhasil maksimal 5 menit setelah presensi pulih. Analisis ATAM menemukan risiko kehilangan pesan (Risiko 1) dan pencatatan ganda (Risiko 2).
Decision	Saat dosen menyetujui pengajuan, Modul Persetujuan menyimpan keputusan dan pesan pencatatan pada tabel outbox dalam satu transaksi Database. Antrean Pesan meneruskan peristiwa PengajuanDisetujui ke Adapter Presensi. Jika gagal, pengiriman diulang dengan jeda bertahap. Setiap pesan membawa kunci idempotensi berupa nomor pengajuan.
Consequences	Positif: persetujuan tidak hilang walau presensi gangguan; pencatatan ganda dicegah; dosen tidak menunggu respons presensi. Negatif: ada komponen dan tabel tambahan, status pesan perlu dipantau, dan penelusuran kesalahan lebih kompleks. Interval retry menjadi sensitivity point yang harus disetel.

BAB IV
PENUTUP

6.1 Kesimpulan

Berdasarkan hasil perancangan arsitektur layanan izin dan sakit mahasiswa, dapat disimpulkan bahwa proses pengajuan izin dan sakit yang sebelumnya dilakukan melalui WhatsApp atau penyerahan surat secara langsung masih memiliki beberapa permasalahan, seperti keterlambatan respons dosen, risiko kehilangan bukti pengajuan, ketidakjelasan status, dan pencatatan presensi yang masih berpotensi dilakukan secara manual.Untuk mengatasi permasalahan tersebut, dirancang sebuah layanan digital yang mengintegrasikan proses pengajuan, unggah bukti, pemeriksaan dosen, persetujuan atau penolakan, notifikasi, serta pencatatan ke sistem presensi.

Arsitektur yang dipilih adalah Modular Monolith dengan pola Layered Architecture dan Event-Driven Architecture. Pemilihan tersebut didasarkan pada kebutuhan sistem yang mengutamakan kemudahan pengembangan, keamanan, keandalan, serta integrasi dengan sistem presensi. Evaluasi menggunakan Utility Tree, analisis trade-off berbobot, dan metode ATAM menunjukkan bahwa rancangan memiliki potensi untuk memenuhi kebutuhan kualitas sistem. Modular Monolith memperoleh nilai 4,15, lebih tinggi dibandingkan Microservices yang memperoleh nilai 3,85 berdasarkan kriteria dan bobot penilaian yang ditentukan.

Dengan demikian, rancangan arsitektur ini diharapkan mampu meningkatkan efisiensi layanan izin dan sakit mahasiswa, mempermudah pengelolaan data, serta meningkatkan transparansi proses presensi di lingkungan Smart Campus UMI.

6.2 Saran

Untuk pengembangan selanjutnya, sistem dapat dilengkapi dengan integrasi langsung terhadap sistem akademik UMI, pengembangan aplikasi berbasis mobile, peningkatan keamanan penyimpanan dokumen, serta pengujian performa dan keandalan sebelum diterapkan secara nyata

DAFTAR PUSTAKA

[1] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA: Addison-Wesley Professional, 2021. Sumber penerbit.

[2] M. Richards and N. Ford, Fundamentals of Software Architecture: An Engineering Approach. Sebastopol, CA: O'Reilly Media, 2020. Sumber penerbit.

[3] R. Kazman, M. Klein, and P. C. Clements, "ATAM: Method for Architecture Evaluation," Software Engineering Institute, Carnegie Mellon University, CMU/SEI-2000-TR-004, 2000. Sumber SEI.

[4] International Organization for Standardization, ISO/IEC 25010:2023 Systems and Software Engineering—Systems and Software Quality Requirements and Evaluation (SQuaRE)—Product Quality Model. Geneva: ISO, 2023. Standar ISO.

[5] OWASP Foundation, Application Security Verification Standard (ASVS) 4.0.3, 2021. [Online]. Available: OWASP ASVS.

[6] M. Nygard, "Documenting Architecture Decisions," Cognitect, Nov. 15, 2011. [Online]. Available: Artikel Documenting Architecture Decisions